

Lezione 16 – Qualificatore const

Corso — Programmazione

Marco Anisetti

e-mail: marco.anisetti@unimi.it

web: <http://homes.di.unimi.it/anisetti/>

Variabili costanti e define

```
#define A 12
```

Non alloca nulla sostituisce. Si tratta di una direttiva del pre-processore

A nel programma ed in memoria non esiste

```
const int A = 12;
```

Il qualificatore **const** avvisa che il contenuto della variabile associata non può essere modificato

Const nel passaggio di parametri

```
void Inizializza(int *t);
```

Riempie l'array passato, quindi deve poterlo modificare

```
void Stampa(const int *t);
```

Stampa l'array quindi non deve poterlo modificare

Nonostante si passi alla funzione un puntatore, quindi un indirizzo di memoria, **la const impedisce la modifica del valore puntato.**

Quindi in sostanza blocca il passaggio per riferimento alla modifica.

Serve solo quando son obbligato ad usare indirizzo ma non voglio davvero modificare

Lezione 16 –Puntatori a Funzione (uso)

Corso — Programmazione

Marco Anisetti

e-mail: marco.anisetti@unimi.it

web: <http://homes.di.unimi.it/anisetti/>

Esempi di uso dei puntatori a funzione

```
#include <stdio.h>
```

```
void fun(int a)
{
    printf("Il valore è%d\n", a);
}
```

```
int main()
{
    void (*fun_ptr)(int) = fun; // &fun

    fun_ptr(10); // (*fun_ptr)

    return 0;
}
```

Array di puntatori a funzione

Rimpiazzo uno switch case

```
#include <stdio.h>

void add(int a, int b) { printf("Somma: %d\n", a+b);}
void sub(int a, int b) { printf("Sottrazione: %d\n", a-b);}
void mul(int a, int b) { printf("Moltiplicazione: %d\n", a*b);}

int main()
{
    void (*fun_ptr_arr[])(int, int) = {add, sub, mul};
    int ch, a = 15, b = 10;

    printf(" 0 add, 1 sub e 2 mul \n");
    scanf("%d", &ch);

    if (ch > 2) return 0;

    (*fun_ptr_arr[ch])(a, b);

    return 0;
}
```

Funzioni passate a funzioni

```
#include <stdio.h>

void fun1() { printf("Fun1\n"); }
void fun2() { printf("Fun2\n"); }

void wrapper(void (*fun)())
{
    fun();
}

int main()
{
    wrapper(fun1);
    wrapper(fun2);
    return 0;
}
```

Ordinare un array di elementi

- Cosa serve per ordinare un array?

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare
 - Se sono numeri basta l'operatore >
 - **Se fossero stringhe?**

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare
 - Se sono numeri basta l'operatore $>$
 - Se fossero stringhe?
 - **Servirebbe un operatore per le stringhe**
 - Funzione *strcmp(s1, s2);*

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare
 - **Applicare il confronto al fine di ordinare**

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare
 - Applicare il confronto al fine di ordinare
 - **L'applicazione dipende dal modo con cui confronto?**

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare
 - Applicare il confronto al fine di ordinare
 - **L'applicazione dipende dal modo con cui confronto?**
 - **NO!**

Ordinare un array di elementi

- Cosa serve per ordinare un array?
 - Sapere come confrontare le cose da ordinare
 - Applicare il confronto al fine di ordinare
- Posso scrivere quindi un modo per ordinare un array generico?

qsort

Ordina array di qualsiasi (*void **) cosa in qualsiasi modo:

Parametro: *int (*comp)(const void *, const void*)*

```
void qsort(void *base, size_t n items, size_t size, int  
(*compar)(const void *, const void*))
```

...

```
int compare (const void * a, const void * b)  
{ return ( *(int*)a - *(int*)b ); }
```

```
int main () {  
    int arr[] = {1, 5, 10, 22, 91, 85};  
    int n = sizeof(arr)/sizeof(arr[0]), i;  
    qsort (arr, n, sizeof(int), compare);  
    .  
    ...  
}
```


Search

```
#include <stdbool.h>
bool compare (const void * a, const void * b){
return ( *(int*)a == *(int*)b );}

int search(void *arr, int arr_size, int ele_size, void *x,
           bool (*compare) (const void *, const void *)){

    char *ptr = (char *)arr; //char * va bene perchè punta a un byte
    int i;
    for (i=0; i<arr_size; i++)
        if (compare(ptr + i*ele_size, x))
            return i;
    return -1;
}

int main(){

    int arr[] = {2, 5, 7, 90, 70}, int x = 7;
    int n = sizeof(arr)/sizeof(arr[0]);
    printf ("Indice %d ", search(arr, n, sizeof(int), &x, compare));
    return 0;
}
```

Parametrizzare una funzione (esercizio 1)

Scrivere un programma che applichi una funzione specifica ad un array di interi

Fare in modo che la funzione da applicare sia un parametro a sua volta

Soluzione(1)

```
5 void Simpleapply(int* things, int (*f)(int), int length);
6 int next(int number);
7 int pot2(int number);
8
9 int main(int argc, char *argv[]) {
10     int a []={ [0]: 1, [1]: 2, [2]: 3, [3]: 4, [4]: 5, [5]: 6, [6]: 7, [7]: 8, [8]: 9};
11     int N=9;
12     int i=0;
13     printf("Array iniziale\n");
14     for(i = 0 ; i < N ; i++){
15         printf("%d,",a[i]);
16     }
17     Simpleapply( things: (int*) a, f: next, length: N);
18     // plot array
19     printf("\nArray dopo la seconda apply\n");
20     for(i = 0 ; i < N ; i++){
21         printf("%d,",a[i]);
22     }
23     Simpleapply( things: (int*) a, f: pot2, length: N);
24     printf("\nArray dopo la seconda apply\n");
25     for(i = 0 ; i < N ; i++){
26         printf("%d,",a[i]);
27     }
```

Soluzione(2)

```
33  void Simpleapply(int* things, int (*f)(int), int length){
34
35      int i=0;
36      for(i = 0; i < length; i++){
37          things[i]=(*f)(things[i]);
38      }
39      return;
40  }
41
42  int next(int number){
43      return number+1;
44  }
45
46  int pot2(int number){
47      return number*number;
48  }
```

Funzione come campo (esercizio 2)

Scrivere un programma che gestisca una entità studente permettendo di calcolare la media dei voti acquisiti.

Considerare uno studente come definito da nome, cognome matricola e un array di voti

Soluzione(1)

```
4 //typedef double (*funzione)(int*, int);
5 typedef struct {
6     char name[10];
7     char surname[20];
8     int votes[3];
9     double (*f)(int*, int);
10 //funzione f;
11 } students;
12
13 double avg (int* a, int b);
14
15 int main() {
16     students s;
17     strcpy(s.name, "Marco");
18     strcpy(s.surname, "Anisetti");
19     s.votes[0]=18;
20     s.votes[1]=20;
21     s.votes[2]=21;
22     s.f=avg;
23
24     printf("La media di %s %s è:%lf\n", s.name, s.surname, s.f(s.votes, 3));
25     return 0;
26 }
```

Soluzione(2)

```
28  ↔ double avg (int* a, int length){  
29      int i=0;  
30      int sum=0;  
31      for (i=0;i<length;i++) {  
32          sum += a[i];  
33      }  
34      return (double)sum/length;  
35  
36  }
```

Funzioni ritornate da funzioni

Il compilatore non supporta direttamente come tipo di ritorno un puntatore a funzione

Dobbiamo usare un typedef

```
typedef return_type (*function_pointer_name)  
(argument_type_1, argument_type_2, .....,  
argument_type_n);
```

```
typedef int (*ptr)(int*);
```

```
typedef ptr (*pm)();
```


Funzioni ritornate da funzioni

Il compilatore non supporta direttamente come tipo di ritorno un puntatore a funzione

Dobbiamo usare un typedef

```
typedef return_type (*function_pointer_name)  
(argument_type_1, argument_type_2, .....,  
argument_type_n);
```

```
typedef int (*ptr)(int*); // tipo ptr puntatore a funzione
```

```
typedef ptr (*pm)(); // tipo pm puntatore a funzione che  
torna ptr
```

Variante esercizio 2

Tramite il typedef posso definire un tipo per la funzione e poi un campo di quel tipo nella struttura

```
typedef double (*funzione)(int*, int);  
typedef struct {  
    char name[10];  
    char surname[20];  
    int votes[3];  
    //double (*f)(int*, int);  
    funzione f;  
} students;
```

Esempio

```
int fun1(int* y) {  
    printf("World\n");  
    return *y + 10;  
}
```

```
ptr fun() //ritorna un tipo puntatore a funzione ptr  
{  
    printf("Hello ");  
    return &fun1;  
    // oppure  
    //int(*pt)(int*)=fun1;  
    //return pt;  
}
```

```
int main() {  
    int a = 10;  
    pm u = fun;  
    printf("%d", (*u())(&a));//??  
    return 0;}  
  
typedef int (*ptr)(int*);  
typedef ptr (*pm)();
```

Esempio

```
int fun1(int* y) {  
    printf("World\n");  
    return *y + 10;  
}
```

```
ptr fun() //ritorna un tipo puntatore a funzione ptr  
{  
    printf("Hello ");  
    return &fun1;  
    // oppure  
    //int(*pt)(int*)=fun1;  
    //return pt;  
}
```

```
int main() {  
    int a = 10;  
    pm u = fun;  
    printf("%d", (*u())(&a)); // ritorna una funzione che chiamo  
    return 0;}
```

*typedef int (*ptr)(int*);
typedef ptr (*pm)();*

Esempio 2

```
int main()
{
    int a = 10;
    pm u = fun;

    int (*(p[1])())(int*) = { u };// ?????

    printf("%d", (*p[0]())(&a));
}
```

Esempio 2

```
int main()
{
    int a = 10;
    pm u = fun;
```

```
int (*(p[1])())(int*) = { u };// è una dichiarazione
```

```
printf("%d", (*p[0])(&a));
}
```

Esempio 2

```
typedef int (*ptr)(int*);
```

```
typedef ptr (*pm)();
```

...

```
int (*(p[1])())(int*) = { u };
```

Assegna u che è di tipo pm ad...

Esempio 2

```
typedef int (*ptr)(int*);
```

```
typedef ptr (*pm)();
```

...

```
int (*(*p[1])())(int*) = { u };
```

Assegna u che è di tipo pm ad...

Un array di dimensione 1 che ha come elemento un...

Esempio 2

```
typedef int (*ptr)(int*);
```

```
typedef ptr (*pm)();
```

...

```
int (*(*p[1])())(int*) = { u };
```

Assegna u che è di tipo pm ad...

Un array di dimensione 1 che ha come elemento un...
function pointer...

Esempio 2

```
typedef int (*ptr)(int*);
```

```
typedef ptr (*pm)();
```

...

```
int (*(*p[1])())(int*) = { u };
```

Assegna u che è di tipo pm ad...

Un array di dimensione 1 che ha come elemento un...

function pointer...

Ad una funzione che ritorna un puntatore a una funzione

Lezione 16 – Qualificatore static

Corso — Programmazione

Marco Anisetti

e-mail: marco.anisetti@unimi.it

web: <http://homes.di.unimi.it/anisetti/>

Variabili static

Una variabile statica locale ad una particolare funzione ha lo scope come le variabili automatiche (locale alla funzione) ma:

- inizializzata una sola volta (la prima volta che tale funzione viene chiamata)
- il suo valore resta inalterato quando si esce dalla funzione
 - quando si richiama nuovamente la funzione la variabile ha ancora il valore assegnatogli precedentemente.
- Dichiarazione

static <tipo> <nome>;

Preserva il valore fuori dal suo scope

Usi

Posso contare quante volte una funzione viene effettivamente chiamata incrementando una variable static locale alla funzione

- **Questo significa che non è allocata sullo stack ma sul segmento dati esterno allo stack**

Si possono inizializzare solo con costanti letterali (mettendoci direttamente un valore)

Funzioni static

Le funzioni sono a scope globale per definizione e sono extern anche

Se le dichiaro static il loro scope si limita al file in cui sono definite. Quindi non posso chiamarle fuori dal file

```
static void fun(){}
```

Usi

Restringere l'accesso al solo file

Riusare lo stesso nome di funzione in altri file

Esempio

In file1.c

```
static void fun1(void)
{
    puts("fun1 called");
}
```

In file2.c

```
int main(void)
{
    fun1();
    getchar();
    return 0;
}
```

Link in compilazione: *gcc file2.c file1.c*

Otengo un errore di funzione non definita per via del fatto che è static

Variabili globali e statiche

- Variabili globali accessibili ovunque
- Variabili statiche hanno lo scope limitato al file in cui sono dichiarate